

FRED REFERENCE SUMMARY

Below we summarize the commands for FRED and their default line addresses. "(.)" means that a command will operate on the current line if no address is specified. "(.,)" means that a command will operate on the current line if no addresses are specified, and on the single addressed line if one address is specified.

Commands which do not require a <nl> as part of their format may be placed one after the other on the same line. Such commands may be separated by one or more spaces; sometimes no spaces are necessary.

Placing an item in square brackets (e.g. "[item]") in the formats below means that the item is optional. Separating items with or-bars as in "item1|item2|item3" means that the user has the choice of any of the given items.

Commands:

A) `(.)A<text>` Append text after line

- must be space or <nl> after the "A"
- text must be ended with \F unless on same line as A and O-IF in effect
- "." set to last line of text typed in

B)	B<bufname>	Buffer
----	------------	--------

- creates buffer if it doesn't already exist
- "." is set to the same position as the last time you were in this buffer

C) (.,.)C<text> Change lines

- must be space or <nl> after "C"
- text must end with \F unless on same line as C and O-IF in effect
- no address wraparound
- "." is set to last line of input text

D) (.,.)^D Delete lines

- no address wraparound
- "." set to line after last deleted

E) E<patname>/<pattern>/ Name pattern

- recursive patterns allowed
- "E<patname>" command turns off name

```
F)      F      Print Facts
```

- prints name, ".", "\$", associated file name, questionable status, of current buffer

F <filename> Set buffer filename

- associates <filename> with current buffer
- marks buffer as questionable

```
FB          Print buffer facts
```

- displays name, ".", "\$", associated file name, questionable status, of each buffer

```
FE                                Print pattern facts
```

- prints all defined patterns in current option format

FF Print buffer file facts

- prints name and associated file name for each buffer

FN Print number registers

- prints name and value of each number register
- when register has a base, prints one N command for base and another for value

FO Print option facts

- prints all options currently turned on

FV Print Version

- prints date this version of the editor was compiled and date the version expires
- a version of the editor cannot be used after its expiry date

F? Show questionable buffers

- shows current buffer and file name, plus names and file names of all questionable buffers

G) (1,\$)G[~]/<pattern>/<commands><nl> Global

- copies commands into hidden buffer and executes them for each line with a string matching <pattern>
- if "~" precedes <pattern>, executes hidden buffer for lines NOT matching <pattern>
- no address wraparound
- sets // to <pattern> at beginning of each iteration
- count register set to number of lines matching <pattern>
- "." set to position it had after the last command executed

I) (.)I<text> Insert text before line

- must be a space or <nl> after "I"
- text must end with \F unless on same line as I and O-I\F in effect
- "." set to last line inserted

J) JM <remarks><nl> Jump Message
JM/<remarks>/

- must be space between "JM" and remarks in first form

JP <remarks><nl> Jump Prompt
JP/<remarks>/

- must be space between "JP" and remarks in first form

JE <remarks><nl> Jump Exit
JE/<remarks>/

- must be space between "JE" and remarks in first form
- if "O+Q", JE will terminate editor
- must be executed from buffer

JO[T|F] Jump Out

- jumps out of executing buffer
- will jump out of one iteration of G and U command list
- JOT jumps out if condition register TRUE, JOF if condition register FALSE
- must be executed from buffer

JB[T|F] Jump Break

- breaks out of any currently executing U command
- JBT breaks out if condition register TRUE, JBF if condition register FALSE

J<label>[T|F]]<commands><nl> Jump to label

- jumps to "@<label>" in current buffer
- in G and U commands, searches for <label> in hidden buffers only
- stream directives accepted in <label>
- stream directives not expanded in <label> search
- J<label>T jumps if condition register TRUE, J<label>F if condition register FALSE
- error if <label> not found

J[T|F] Jump line

- JT skips rest of line if condition register TRUE, JF skips rest of line if condition register FALSE

K) (.,.)K<bufname> Kopy lines to buffer

- moves addressed lines to <bufname>, copies them back to current buffer
- deletes previous contents of <bufname>
- no address wraparound
- concatenates lines without <nl> characters
- sets "." in <bufname> to last line, sets "." in current buffer to last line Kopied

L) L[<filename>] List file on terminal

- does not change either "." or file name associated with buffer

M) (.,.)M<bufname> Move lines to buffer

- deletes previous contents of <bufname>
- no address wraparound
- sets "." in <bufname> to last line, sets "." in current buffer to "." line after last line moved

N) (.)N<regname>:<number> Number (assign)
 N<regname>| (absolute value)
 N<regname>- (switch signs)
 N<regname>+<number> (add)
 N<regname>-<number> (subtract)
 N<regname>*<number> (multiply)
 N<regname>/<number> (divide)
 N<regname>%<number> (remainder)
 N<regname>~ (bit complement)
 N<regname>&<number> (bitwise AND)
 N<regname>|<number> (bit inclusive OR)
 N<regname>^<number> (bit exclusive OR)
 N<regname>}<number> (right bit shift)
 N<regname>{<number> (left bit shift)
 N<regname>=<number> (test equality)
 N<regname><<number> (test less than)
 N<regname>><number> (test greater than)
 N<regname>B<number> (set base)
 N<regname>D<number> (set digits)
 N<regname>F<char> (set fill char)
 N<regname>P (print)
 N<regname>A (assign line address)
 N<regname>L (count characters)

- all N commands set count register to value in <regname>
- numbers may be positive or negative integers
- may also use special characters as numbers: "#" count register value, "\$" line number of last line in current buffer, "." line number of current line
- arithmetic operations can be combined

O) O[+|-]<opt-char>[<parameters>] Options

- are turned on or off

Input options

O[+|-]I((parentheses on names)

- default is O+I(, parentheses always required

O[+|-]IL<char> (lower case input)

- upper case characters must be preceded by <char>

O[+|-]IU<char> (upper case input)

- lower case characters must be preceded by <char>

O[+|-]IE<char> (set escape char)

- default is "O+IE\"

O[+|-]IF (print linefeed on return)

- O-IF doesn't print linefeed when user enters carriage return
- default is O+IF (linefeed is printed)

O[+|-]IA/<string/> (set text input prompt)

- default is O+IA/>/

O[+|-]IC/<string>/ (set command prompt)

- default is O-IC

O[+|-]IR/<string>/ (string after return)

- string is output after every carriage return or break
- default is O-IR

O[+|-]I\F (is \F necessary?)

- default is O-I\F

Output Options

O[+|-]OO (print oddities in octal)

- default is "O+OO"

O[+|-]OL (lower case output)

- upper case characters are preceded by case escape character

O[+|-]OU (upper case output)

- lower case characters are preceded by case escape character

O[+|-]O<chars> (alt. output format)

- for terminals lacking some recognized characters

String Options

O[+|-]SD (ignore case in matches)

- default is "O+SD"

O[+|-]S& (& and {}x in S command)

- default is "O+S&"
- enabled special meanings can be disabled with \C, disabled special meanings can be enabled with \O

O[+|-]S<chars> (pattern chars)

- default is "O+S\$.*^\[E", "O-S{(|+#@"
- condition register is set TRUE if any <chars> previously had special meanings on, FALSE otherwise

O[+|-]S/ (is final "/" needed?)

- applies to line searches, E, G, JE, JM, JP, S, T, ZL, ZT, ZU
- default is O+S/

Other Options

O[+|-]P<number> (set pagesize)

- default is "O-P"

O[+|-]Q (quit option)

- default is "O-Q"

O[+|-]B<number> (in-core pages)

- default memory size is three pages of 640 words each
- "O+B<number>" requests an additional <number> pages
- "O-B<number>" releases <number> pages provided they are unused

O[+|-]C/<str1>/<str2>/ (translate a string)

- translate <str1> into <str2> wherever it appears in input
- intended for translating function keys into useful strings
- O-C cancels last translation
- O-C/<str1>/ cancels translation of <str1> (but need to write <str1> in some untranslatable way)

O[+|-]R (default read perms)

- O+R indicates that FRED should put general read permissions on all files that it creates; O-R dictates no general permissions
- default is O+R

O[+|-]T<tab-list> (set tabs)

- default tabs are "O+T5,9"
- O-T suppresses tab display

O[+|-]M (monitor)

- O+M provides trace-type output of executing buffers
- default is "O-M"

O[+|-]W (set line width)

- O+W<number> tells FRED that lines wrap around on the screen in column <number>; O-W tells FRED that each line of text counts as one line on the screen

P) (.,.)P	Print
(.,.)<n1>	Print addressed lines
<n1>	Print ".+1"

- "." is set to last line printed

Q) Q<n1> Quit

- quits provided there are no questionable buffers and current buffer does not contain unwritten text

QQ<n1> Quit despite quest. buffer

- quits regardless of questionable buffers

Q!<TSS command><n1> Quit with system call

- quits regardless of questionable buffers
- uses "drl gotoss" to call TSS system

R) (0)R[X][<filelist>]<n1> Read file

- <filelist> consists of list of file names, separated by commas; trailing comma allowed
- FRED tries to read first file in list; goes on to next if file can't be found or file name has syntax error; keeps going through entire list until it finds file or end of list
- condition register set to TRUE if a read operation works; otherwise FALSE
- if FRED gets through list and finds trailing comma, condition register set to TRUE, count register set to -1
- will use file name associated with current buffer if <filelist> not specified
- "RX" prints statistics of read (number of blocks, lines, and characters read, plus buffer and file name) even when program is executing from a buffer
- "." is set to last line read
- count register set to number of blocks read

(0)R[X]<start>[<filelist>]<n1>

- <start> is positive integer
- same as previous R command, except that the read operation will start at line <start> in the first file found

(0)R[X]<start>-<finish>[<filelist>]<n1>

- <start>, <finish> are both positive integers
- same as previous R command, except that the read operation takes lines from <start> to <finish> in the first file found

(0)R[X]<start>,<count>[<filelist>]<n1>

- <start>, <count> are both positive integers
- same as previous R command, except that the read operation begins at line <start> and reads in a total of <count> lines from the first file found

S) (.,.)S/<pattern>/<string>/ Substitute

- no address wraparound
- any non-alphanumeric delimiter is accepted in place of "/"
- "." is set to the last line where substitutions are made
- count register is set to the number of substitutions made
- if no substitutions made condition register is set to FALSE, otherwise TRUE
- error issued if no substitutions made and command was not executed from buffer

T) (.,.)T[~]/<pattern>/ Test for pattern

- condition register set to TRUE if pattern found, otherwise FALSE
- if "~" precedes <pattern>, condition register set to FALSE if pattern found, otherwise TRUE
- if first address less than second, search goes forward through buffer from first to last; if first address greater than second, search goes backward through buffer from first to last

U) (.)U[<number>][T|F|E] <commands><nl> Repeat Until

- "U<number>" repeats commands <number> times
- "UT" repeats commands till condition register is TRUE at end of an iteration
- "UF" repeats commands till condition register is FALSE at end of an iteration
- "UE" repeats commands till error occurs
- if <number> is specified with "UT", "UF", or "UE", commands are repeated <number> times unless condition is satisfied first
- commands listed are copied into a hidden buffer which is then executed repeatedly until the given condition is satisfied
- U commands may be nested to any level

V) V Void last active line

- voids the effects of last line containing an active command; active commands are a, b, c, d, e, i, k, m, n, o, r, s, za, zb, zd, zi, zk, zl, zm, zs, zt, zu, zv, and f and w when they specify a new file name.
- up to 10 levels of voiding
- to void V, use ZV

W) (1,\$)W[X][A|B][<filename>]<nl> Write file

- if no <filename> specified, file name associated with the current buffer is used
- WX prints statistics for a write (number of blocks, lines, and characters written, plus buffer and filename) even when command was executed from a buffer
- WA forces writing in ASCII; WB forces writing in BCD; if neither A nor B specified, file will be written in format of file associated with the current buffer; if no such file, default is ASCII
- "." not changed by W
- count register set to number of blocks written

Y) Y Why error?

- prints full description of most recent error

Z) (.)Z Set address

- sets current line pointer "." to specified address

(.)ZA<bufname> Append buffer

- appends buffer after "." in current buffer
- "." is set to last line appended

(.,.)ZB Sort into descending order

- condition register is set to TRUE if the lines sorted were all the same, FALSE otherwise

ZC Clear questionable flag

- condition register is set to FALSE if flag was not on; condition register is set to TRUE if flag was on (i.e. buffer was questionable)

ZD<bufname> Delete buffer

- clears contents, associated file name, questionable flag

ZG<bufname><commands><nl> Gather output

- executes given commands
- output normally sent to terminal is appended after "." in buffer <bufname>
- no terminal processing done on gathered output
- anomalous results if <commands> modify <bufname>

(.,.)ZI Zap in tabs

- replaces spaces with tabs if possible
- no address wraparound

(.,.)ZK<bufname> Zap Kopy

- Copies lines in current buffer into <bufname>, appending them after "." in <bufname>
- previous contents of <bufname> stay intact
- <bufname> can be current buffer, but addressed lines cannot include "."
- "." is set to last line copied in both <bufname> and current buffer

(.,.)ZL[/<pattern>/] Zap Lower case

- ZL puts all letters into lower case; count register is set to number of lines converted
- ZL/<pattern>/ puts all strings matching <pattern> into lower case; count register is set to number of strings converted
- no address wraparound
- "." is set to last line where characters were converted

(.,.)ZM<bufname> Zap Move

- moves lines out of current buffer and into <bufname>, appending the lines after "." in <bufname>
- previous contents of <bufname> stay intact
- <bufname> can be current buffer, but addressed lines cannot include "."
- "." in <bufname> is set to last line moved in, "." in current buffer set to line after last line moved out

(.,.)ZO Zap out tabs

- replaces tabs with spaces
- no address wraparound

ZP<bufname> Print buffer

- prints contents of <bufname> on terminal

ZR <filename> Go to buffer with file

- if there is a buffer with associated <filename>, go to that buffer
- if <filename> is not associated with any buffer, create new buffer and read in file
- new buffer names are b(a), b(b), etc., then b(~0), b(~1), b(~2),...
- if read actually happens, ZR prints out stats in form used by R command; if just buffer switch, ZR prints out stats in form used by F command

(.,.)ZS Sort into ascending order

- condition register is set to TRUE if the lines sorted were all the same, FALSE otherwise

(.,.)ZT/<string1>/<string2>/ Translate

- the characters in <string1> are converted to corresponding characters in <string2>
- <string1> must be same length as <string2>
- no characters can appear more than once in <string1>
- count register is set to number of characters translated
- "." is set to last line where translations made
- no address wraparound

(.,.)ZU[/<pattern>/] Zap Upper case

- ZU puts all letters into upper case; count register is set to number of lines converted
- ZU/<pattern>/ puts all strings matching <pattern> into upper case; count register is set to number of strings converted

- no address wraparound
- "." is set to last line where characters were converted

ZV Void most recent V

- undoes most recent V command, provided there have been no intervening lines containing active commands

(1,\$)ZW<write options> Write append

- appends addressed lines on the end of a file
- otherwise takes same options as W command, works in same way

!) !<TSS command> Perform TSS command

- issues "drl callss" to TSS
- condition register set to TRUE if the call terminates normally; condition register set to FALSE if the call is terminated by "break"

") (.)"<comment><nl> Comment

- FRED ignores rest of line

#) # Print count register
=) (.)= Print line #

- prints the line number of addressed line; "." is not changed

:) (.):[<number>] Print window

- prints window of <number> lines before and after specified address; if no <number>, five is used (so ":" is equivalent to "-5,+.5p")
- "." is set to last line printed

\) \[<number>] Wait

- if <number> supplied, goes to sleep for <number> seconds
- if no <number>, goes to sleep for an hour
- maximum sleep, 24 hours

Stream Directives

\B<bufname>

Inserts contents of buffer <bufname> in input stream. May be used recursively.

\C<char>

Causes <char> to be taken literally.

\E<patname>

Represents pattern previously associated with <patname> by an E command. Only special inside a pattern.

\L<bufname>

Similar to \B<bufname> except that all the characters in <bufname> are considered to be preceded by a \C.

\N<regname>

Places the contents of number register <regname> in the input stream, according to any applicable register format commands.

\O<char>

Causes <char> to take on any special meaning it can have in this context, regardless of current option settings.

\R

Takes one line of input from the terminal. No stream directives on this line are effective. The <nl> character at the end of the line is discarded.

\S<bufname>

Similar to \L<bufname> except that all <nl> characters in <bufname> are removed as the buffer is placed into the input stream. Special pattern characters in <bufname> are taken literally, without their special meanings.

\W

Replaced by the file name associated with the current buffer.

File I/O

The W command will create files it cannot find. If the specified file name is less than nine characters and does not contain a "/" or "\$", a temp file is created; otherwise a permanent file is created.

Permanent files are removed from the AFT after every successful read or write; temp files are not removed from the AFT.

When a permanent file is read or written without a specified line address, the FULL name of the file is associated with the current buffer. The file name associated with the current buffer is not changed if line addresses are specified. If a file with a password is read or written without specified line addresses, FRED will associate the name of the file with the current buffer, but will strip away the password of the file. Thus the user must specify the password every time the file is read or written, since FRED will not remember the password.

If an I/O error occurs in accessing a file, the Y command can be used to obtain an explanation of why the access failed.

Patterns and the Pattern Matcher

Special Characters:

^

This character matches the null character beginning a line.

\$

This character matches the null character before the <nl> in lines which have a <nl>. In lines without a <nl>, "\$" matches the physical end of line.

.

This character matches any character except <nl>.

+

The construction "<pattern>+" matches any sequence of one or more strings matching <pattern>.

*

The construction "<pattern>*" matches any sequence of zero or more strings matching <pattern>.

@

The construct "@(<n>)" matches the null string immediately before the <n>th character in a line. "@(-<n>)" matches the null string immediately after the <n>th character from the end of a line. "@(-1)" matches the null string immediately before the <nl>. "@(0)" matches the null string immediately after the <nl>. "@(<n>-)" matches the null string before the <n>th character in a line or before any previous character in the line. "@(<n>+)" matches the null string before the <n>th character in a line or before any subsequent character in the line.

|

The construction <pattern1>|<pattern2> matches any string which matches either <pattern1> or <pattern2>.

()

Any pattern may be placed in parentheses to obtain a pattern which matches the original.

[

"[<string>]" defines a pattern which matches any character in <string>. "[^<string>]" defines a pattern which matches any character NOT in string.

{}

The construction {<pattern>}<char> creates a pattern in which any string matching <pattern> is associated with the name <char>. <char> may be used to represent the matching string on the right hand side of S commands.

\E

See section on Stream Directives.

#

The "#" prevents the Pattern Matcher from backing up and making a second attempt to find a string matching a given pattern.

//

The null pattern // matches the pattern most recently encountered.

NULL

The null string (generally written () to distinguish it from the null pattern //) matches any null character.

The construction <number>/<pattern>/ defines a quasi-pattern which matches the <number>th occurrence of <pattern> in a line. The construction -<number>/<pattern>/ defines a quasi-pattern which matches the <number>th occurrence of <pattern> from the end of a line. Quasi-patterns may be used in the place of patterns in E, S, ZL, and ZU commands.

Line Addresses

Valid Forms:

<number>

Ordinary line number.

\$

Last line in buffer.

.

Current line in buffer.

+<number>

As first address, equivalent to "+<number>". As second address in address pair, equivalent to first address plus <number>. If no <number>, equivalent to "+1".

-<number>

As first address, equivalent to "-<number>". As second address in address pair, equivalent to first address minus <number>. If no <number>, equivalent to "-1".

/<pattern>/

First line after "." containing <pattern>. Search for <pattern> begins at line after ".", wraps around to top of buffer if necessary, ends at "." if <pattern> not found.

~/<pattern>/

First line after "." which does not contain <pattern>.

-/<pattern>/

First line before "." containing <pattern>. Search for <pattern> begins at line before ".", wraps around to bottom of buffer if necessary, ends at "." if <pattern> not found.

-~/<pattern>/

First line before "." which does NOT contain <pattern>.

<addr>

Symbol used for valid addresses above or address constructs below.

<addr>+<number>

Refers to the <number>th line after <addr>.

<addr><number>

Same as <addr>+<number>.

<addr>-<number>

Refers to the <number>th line before <addr>.

<addr>.

Refers to <addr>+1.

<addr>+/<pattern>/
Refers to first line after <addr> containing <pattern>. Search for <pattern> begins at <addr>+1, wraps around to the top of the buffer if necessary, and ends at <addr> if <pattern> is not found.

<addr>/<pattern>/
Same as <addr>+/<pattern>/.

<addr>~/<pattern>/
Refers to first line after <addr> not containing <pattern>.

<addr>-/<pattern>/
Refers to the first line before <addr> containing <pattern>. Search for <pattern> begins at <addr>-1, wraps around to the bottom of the buffer if necessary, and ends at <addr> if <pattern> is not found.

<addr>~/<pattern>/
Refers to first line before <addr> not containing <pattern>.

^
Shorthand for ".-1".

<addr>^
Shorthand for "<addr>-1".

*
Shorthand for "1,\$".

<addr>*
Shorthand for "<addr>,\$".

&
Shorthand for ".+1,.<pagesize>" where <pagesize> is set by the O+P command. If no <pagesize> has been set, & will generate ".+1,.+20".

<addr>&
This generates the pair of addresses "<addr>,<addr>+<pagesize>".

<addr>-&
This generates the single address <addr>-<pagesize>.

Buffer Programs

A program is invoked by the command line, "fred filename arg1 arg2 ..." where filename is the name of the file containing the program. If filename is not found under the current catalog, the FRED bootstrap looks for it under "/fred/filename" and then "fred/filename".

The arguments "arg1", "arg2", etc. (if any) are placed on separate lines in buffer "0". The specified file is read into buffer ".". The date is placed into buffer "d" in the numerical format "mm/dd/yy". The time is placed into buffer "t" in the numerical format "hh:mm". The current userid is placed in buffer "u". If there is an init file named "/fred/.init", FRED will execute those inits before it executes the commands it had read into buffer ".".

Language Limitations and Special Considerations

When reading from a file, FRED will not accept a record of longer than 2000 characters. Lines longer than 2000 characters are broken up into separate lines of 2000 characters each. An S command cannot lengthen any line beyond 2000 characters.

FRED uses the octal character \777 as a special internal character; for this reason, the character may not appear in input text.

If null lines are being supplied to fill a \R construction, and line feeds are used rather than carriage returns, every other line feed is ignored.